

TLS Fingerprinting for Malicious vs Benign Traffic

Christian Alicea (caalicea), Joseph Marino (jzm522), Samuel Juarez(sejuarez)

Project overview

The project involves the use of TLS Fingerprinting as a means of detecting malware or vulnerable applications through analyzing the ClientHello message that is presented during a TLS handshake. The reason machine learning comes into play is due to the fact that attackers can attempt to mask their ClientHello messages to appear as legitimate browsers, though they can never completely hide it. By feeding both benign and malicious traffic to a machine learning model, we were able to achieve highly accurate models that are able to spot the tiny flaws in the masked ClientHello messages, therefore keeping the network secure from any malware and/or bots.

Motivation and Relevance

Nowadays, the vast majority of web traffic is encrypted using TLS or SSL. Encryption plays an essential role in protecting user privacy, but it makes it harder for network defenders to catch malicious traffic. Since, defenders cannot decrypt the traffic, it must rely on how the connection was established.

Before an encrypted connection is established, the client sends a *ClientHello* message. This message contains specific parameters that are unique to the software starting the connection. The specific combination and order of these parameters can be the connection's "DNA." Since malicious actors attempt to disguise malware traffic as normal traffic, they often rely on different networking libraries, which inevitably leak tiny discrepancies.

Differences between a real user and disguised malware are subtle variables in the connection's DNA, which is why static rules for detection often fail. As browsers update, established detection rules suffer from concept drift, making them obsolete. By utilizing a machine learning model for features in datasets, the model can be made to dynamically learn complex patterns to distinguish between benign users and malicious actors.

Methodology:

We set up the EC2 that helped download large data files. That data was then fed to a Google Colab file for preprocessing. The data was loaded and the script parses the data, converts target labels to 0 for benign and 1 for malicious, and then encodes categorical variables to complete the data set. We also removed several columns that we determined not to be useful, like IP address and timestamps. To help our code run faster, we downsized our data to 100,000 entries, followed class procedures and created a 60/20/20 split to begin testing different models and see which performs best with a 50-50 split between malicious and benign traffic.

We constructed 3 models for analysis and comparison:

- Basic Decision Tree
- One Class SVM
- Histogram Based Gradient Boosted Decision Tree

To train the Decision tree model, we varied three parameters, Max Depth, Min Sample Leaf, and Min Sample Split. It works by making a series of splits based on feature thresholds, which creates a flow chart path to a final prediction. The Max depth parameter limits how many levels the tree can grow. The min sample split is the minimum amount of data each branch can have before a split, and the minimum sample leaf is the minimum number of data points that must result in a node.

To train the SVM model, we first took all the malicious data out of the dataset, then we varied the Nu and Gamma parameters to get the best model. Unlike the decision tree models we use, this model tries to make a boundary that holds normal data, then classifies anything outside this boundary as malicious. The Nu parameter is a limit on training errors, and Gamma controls how tightly wrapped the boundary is to the data.

To train the Histogram Based Gradient Boosted Decision Tree, we varied hyperparameters of learning rate, Max Depth, and Min Sample Leaf. This model differs from the basic decision tree for two reasons. The first is the gradient boosting part. It creates many decision trees, and learns based on the mistakes of each sequential tree. This creates an extremely deep model, but requires a lot of processing power and time. This is where the second part comes in, the histogram. This combines feature data into bins (why it is called a histogram), which limits the potential spots where the decision tree can make its splits, thus reducing the processing time. Combining these two parts lets us utilize the speed of the histogram and the power of the gradient boosting. This results in parameters of learning rate, max depth, and min sample leaf. The additional parameter learning rate that isn't in the basic decision tree is essentially how much the model learns from each sequential decision tree.

Results

1. Decision Tree

To find the optimal decision tree model configuration, hyperparameter tuning was conducted across 27 distinct combinations. The parameters tested included maximum tree depth (depth = 4, 6, 8), minimum samples at a leaf node (minleaf = 1, 5, 10) and minimum samples required to split (minsplits = 20, 50, 100). The minimum leaf and minimum splits had minimal impact on the overall performance metrics, while max depth showed a strong influence in the F1 metric. Figure-1 below, shows the performance between the all possible parameters combination, and the best parameters found were depth = 8, minleaf = 10 and minsplits = 100.

F1 vs Parameters

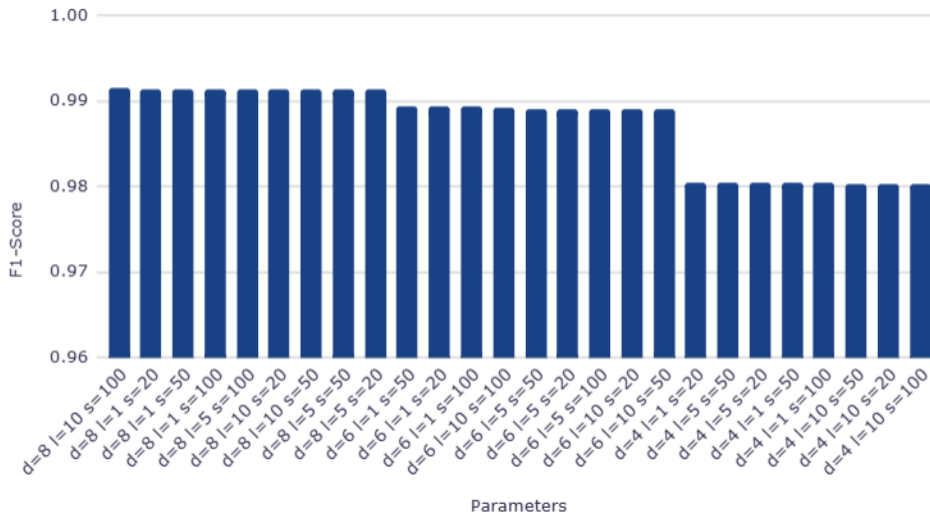


Figure 1: Basic Decision Tree F1 scores for each configuration

2. One Class SVM

For the One-Class SVM model, the tested parameters included NU (0.01, 0.05, 0.1) and Gamma (scale, auto, 0.1). Moderate NU values lead to more flexibility when labeling anomalies, and higher gamma values lead to more localized pattern captures. The model showed sensitivity to tuning with an F1 score between ~0.66 and 0.77. Best parameters found were NU = 0.05 and Gamma = 0.1

F1 vs Parameters

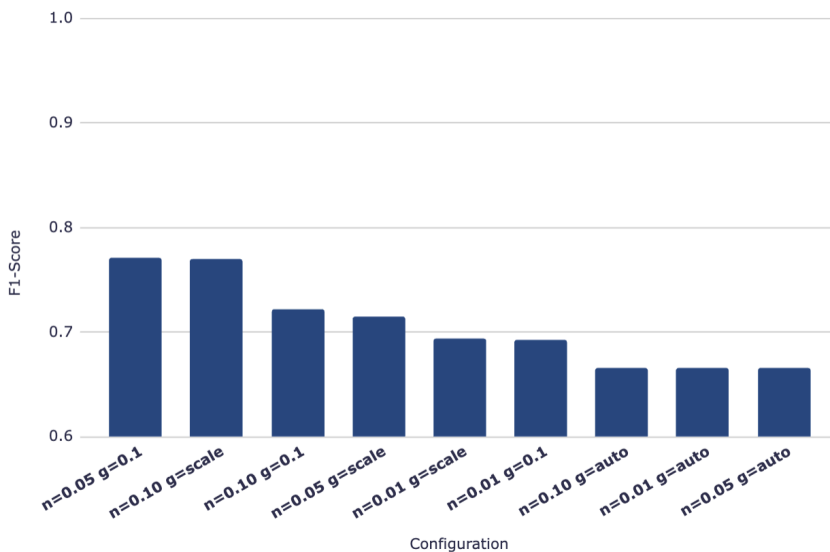


Figure 2: One-Class SVM F1 scores for each configuration

3. Histogram Based Gradient Boosted Decision Tree

For the Histogram Based Gradient Boosted Decision Tree, out of our parameters of Learning rate = [0.05, 0.10], Max depth = [4, 8], and min sample leaf = [20, 100] our best parameters were learning rate = 0.1, depth = 8, and min sample leaf = 20. However, as seen in figure 3, all configurations performed at similar levels. With no clear separation between configurations, we can conclude that the model is more reliant on the process than the parameters, and that it is insensitive to tuning. This is because of how in depth the gradient boosting process is. It has the ability to create a robust model with any amount of depth, learning rate, and sample leaf. Across the model, we saw a F1 score that surpassed .99, similar to the decision tree model.

F1 vs Configuration

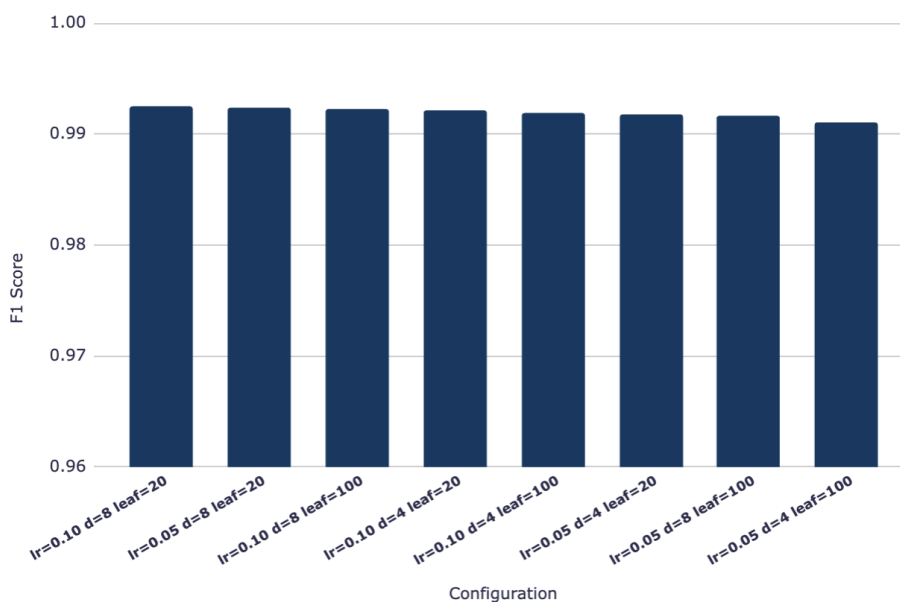


Figure 3: Histogram Based Gradient Boosted Decision Tree F1 scores for each configuration

4. Results comparison Table

As seen in Figure 4, our Basic Decision Tree and Histogram Based Gradient Boosted Decision Tree models both performed extremely well. Each had F1, precision, recall, rocauc, and prauc above .98 for all categories. The SVM performed notably worse, indicating a need for labeled malicious data to detect real malicious data. It however, did extremely well in precision with a .96 score. This makes sense, as the model would minimize false positives since it was trained on all normal data.

```
[{'model': 'Model #1 - Decision Tree',
  'precision': 0.9981753674607197,
  'recall': 0.986080512717805,
  'f1': 0.9920910785350864,
  'rocauc': np.float64(0.9930012162823838),
  'prauc': np.float64(0.9926266441027072)},
 {'model': 'Model #2 - OC-SVM',
  'precision': 0.9663351185921959,
  'recall': 0.6323853394752653,
  'f1': 0.7644815689122935,
  'rocauc': np.float64(0.8958764009177459),
  'prauc': np.float64(0.9140333412930854)},
 {'model': 'Model #3 - Histogram-Based Gradient Boosting',
  'precision': 0.9991897913712781,
  'recall': 0.9879831764470258,
  'f1': 0.9935548841893252,
  'rocauc': np.float64(0.9958009217698066),
  'prauc': np.float64(0.9953002721576616)}]
```

Figure 4: Each models top configuration results across F1, Precision, Recall, ROCAUC, and PRAUC

F1 vs Model

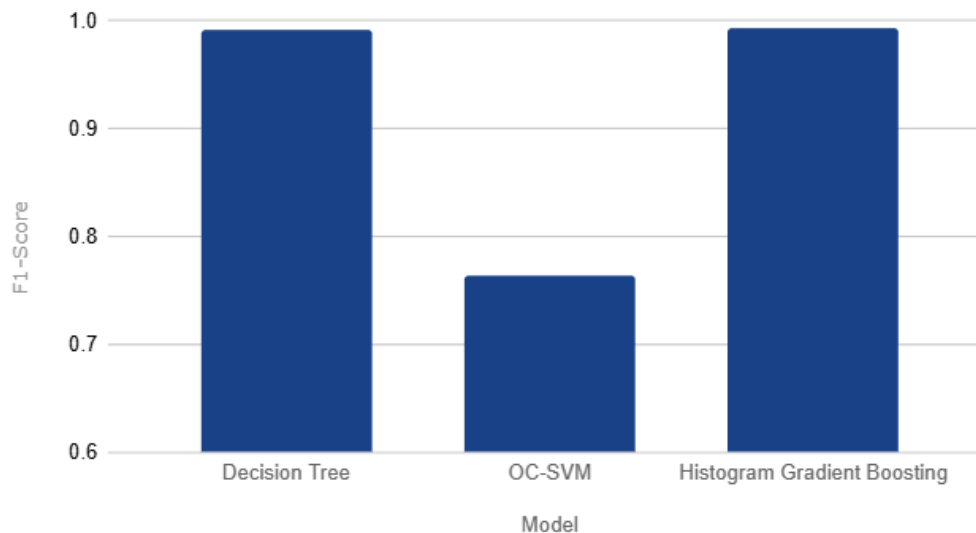


Figure 5: F1 Vs Model Results Comparison

Challenges and limitations

One of the biggest challenges is often the quality and accessibility of raw data. For this project, we have decided to use the IoT-23 dataset which contains data captured in 2018-2019. This will

create issues with protocol obsolescence, since IoT-23 uses very basic, hardcoded TLS implementations or older SSL versions. So we will need to make sure our model does not identify “old TLS” as malicious, leading to false-positives. Another potential problem we may encounter could be overfitting, since IoT devices have small TLS stacks, making their fingerprints very static. The full dataset is over 20GB of compressed data, so parsing these PCAPs to extract TLS features will require significant computational power.

To test against existing models, using the same test data sets would be the best comparison, however most authoring groups don’t allow easily obtainable access to such datasets. We can request access to their preprocessed datasets, or use the separate sources they pulled the data from.

Concept drift would be the biggest challenge to using this model in real applications. The speed at which the datasets and malicious attacks are evolving would cause major issues for our model. Without a solution to feed new labeled data in real time, concept drift would make our model obsolete very quickly.

Future Steps

If we were to continue working on this project, we would want to improve in the following areas:

Introduce a Data Pipeline to extract features from Client Hello messages in real time. We could use a Zeek pipeline to achieve this. It would be a step towards solving the concept drift issue since the data it is fed will evolve our model as fast as the datasets and attacks are evolving.

Improve our Feature Space by adding more fields. The feature space in TLS fingerprinting has evolved to JA3, JA4 and other patterns such as cipher suite patterns. Including these types of data would help to modernize our model, allowing it to catch malicious data in those data formats as well.

Apply more test data to continue to improve our tuning. To better compare our models to ones found in literature, we could take the test datasets they use in order to get a one for one comparison of how our model stacks up against them. This would lead us to find better tuning configurations, thus giving us a better final product.

Works Cited

Sebastian Garcia, Agustin Parmisano, & Maria Jose Erquiaga. (2020). IoT-23: A labeled dataset with malicious and benign IoT network traffic (Version 1.0.0) [Data set]. Zenodo.
<http://doi.org/10.5281/zenodo.4743746>